Alta Disponibilidad con ALB + Auto Scaling + IMDSv2

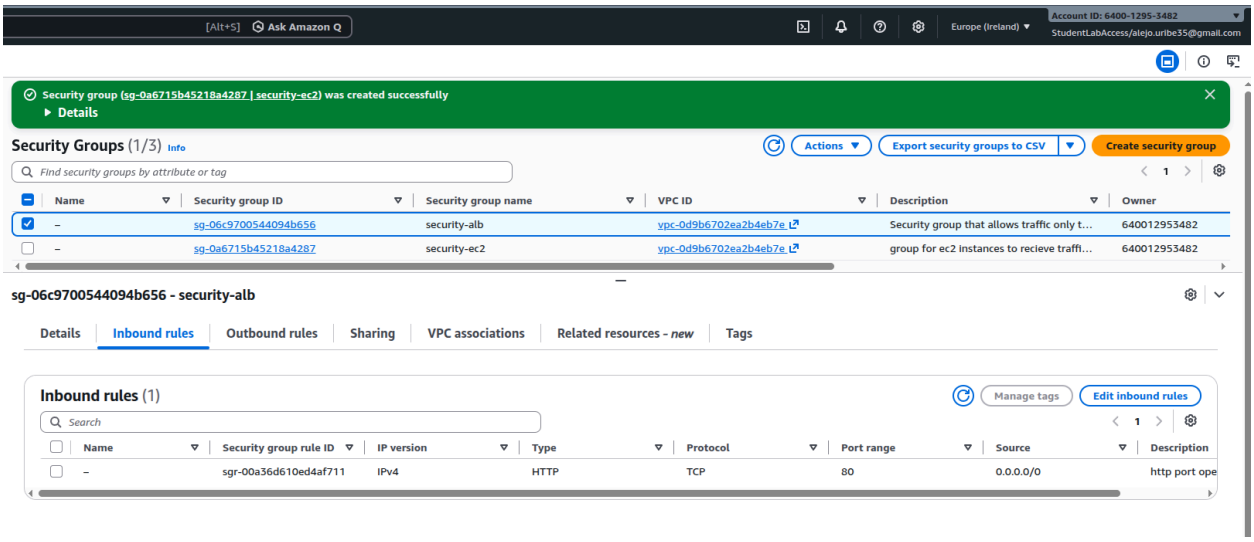
Repositorio

https://github.com/Alejoles/app-commit-lab-alb

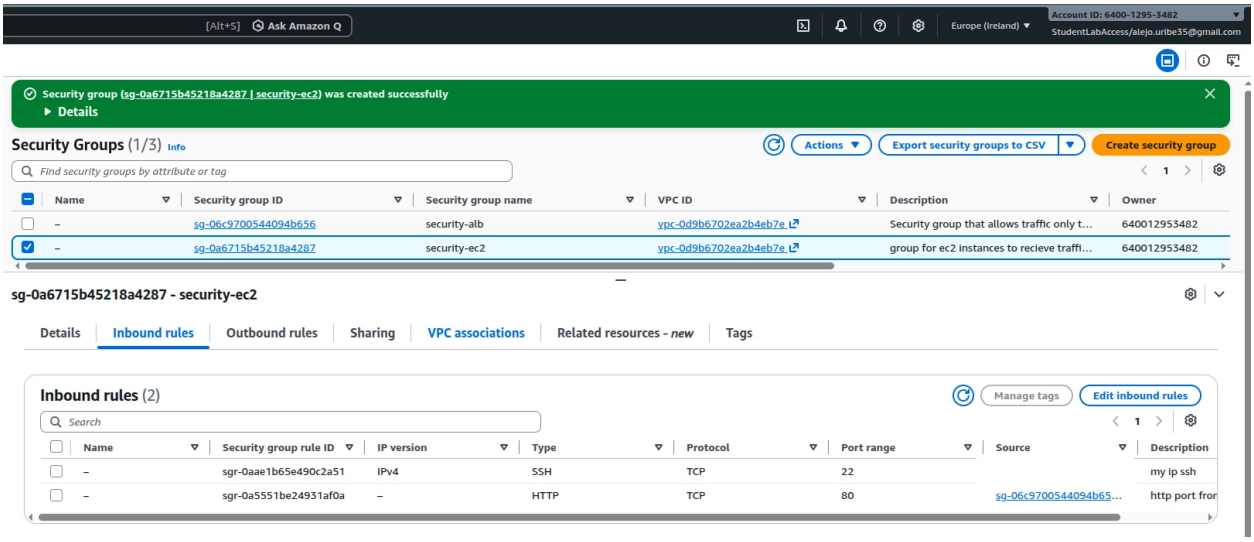
Entregable 1: SGs

📸 **Entregable 1:** Captura mostrando ambos Security Groups y sus reglas.

security ALB



security EC2



Entregable 2: Launch Template

Usaremos este user data, no el que estaba en el taller, en principio deberían servir para algo muy parecido.

```
#!/bin/bash
# Guarda todo lo que ocurre en un log (ayuda a depurar)
exec > >(tee -a /var/log/user-data.log) 2>&1
set -x

# Actualizar paquetes
yum update -y

# Instalar NGINX (amazon-linux-extras solo existe en AL2; en AL2023 entra yum) y Git
amazon-linux-extras install -y nginx1 || yum install -y nginx
yum install -y git

# Iniciar y habilitar NGINX
systemctl start nginx
systemctl enable nginx

# Obtener token IMDSv2
```

```
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token"
\
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600" -s)

# Obtener datos de la instancia usando el token
INSTANCE_ID=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" \
  -s http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" \
  -s http://169.254.169.254/latest/meta-data/placement/availa
bility-zone)
PRIVATE_IP=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" \
  -s http://169.254.169.254/latest/meta-data/local-ipv4)
INSTANCE_TYPE=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" \
  -s http://169.254.169.254/latest/meta-data/instance-type)

# Endpoint de health check
echo ok > /usr/share/nginx/html/health

# Página básica (se queda si no se puede desplegar la app de
GitHub)
cat <<EOF > /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html>
<head>
    <title>ALB + ASG Lab</title>
    <meta http-equiv="refresh" content="2">
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            margin-top: 50px;
            background-color: #f2f2f2;
        }
        h1 { color: #232F3E; }
        h2 { color: #FF9900; }
    </style>
```

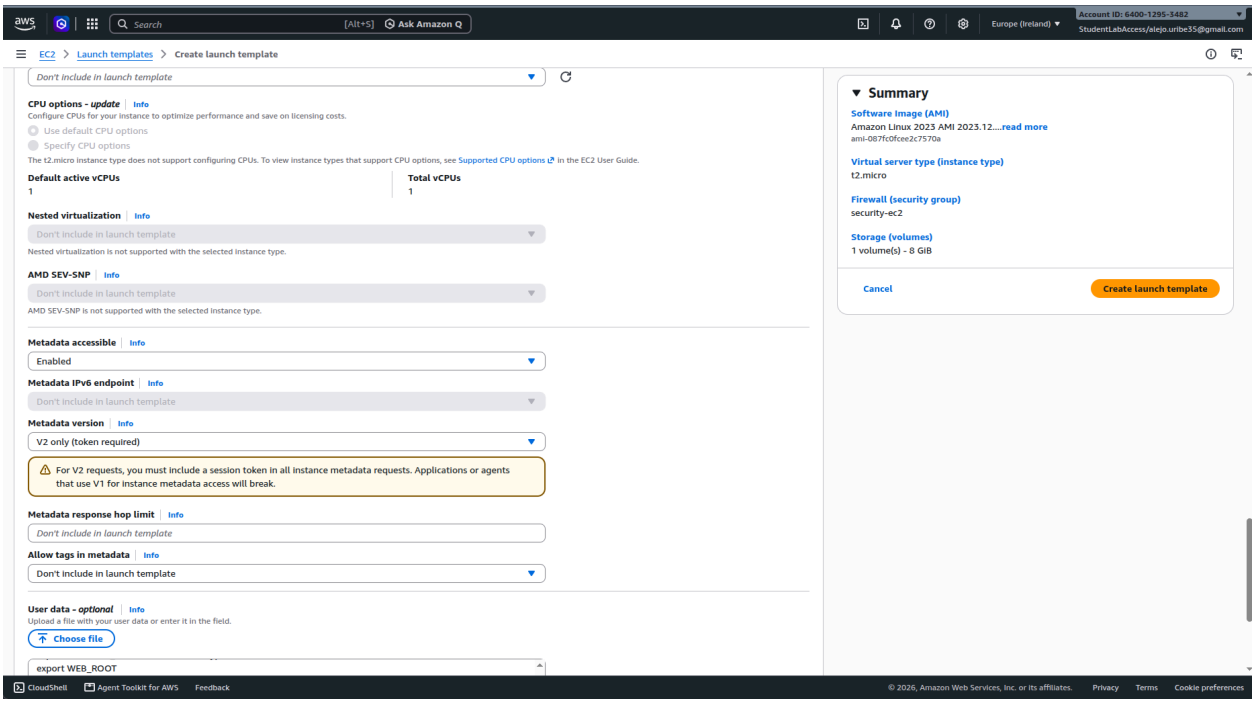
```
</head>
<body>
    <h1>Servidor activo</h1>
    <h2>Instance ID: $INSTANCE_ID</h2>
    <p>AZ: $AZ | IP privada: $PRIVATE_IP</p>
</body>
</html>
EOF

# Desplegar la app desde GitHub (si falla, queda la página básica)
REPO_URL="https://github.com/Alejoles/app-commit-lab-alb.git"
if GIT_TERMINAL_PROMPT=0 git clone --depth 1 "$REPO_URL" /opt/alb-asg-lab; then
  export INSTANCE_ID AZ PRIVATE_IP INSTANCE_TYPE
  export COMMIT=$(git -C /opt/alb-asg-lab rev-parse --short HEAD)
  bash /opt/alb-asg-lab/scripts/render.sh || echo "WARN: render.sh falló, se mantiene la página básica"
else
  echo "WARN: no se pudo clonar el repo, se mantiene la página básica"
fi

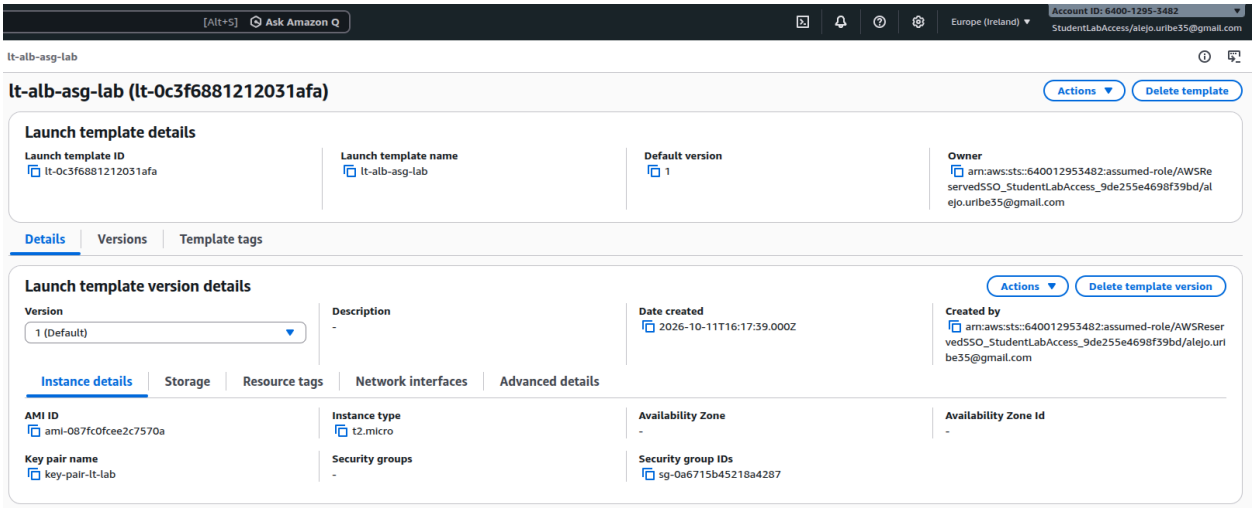
# Reiniciar NGINX
systemctl restart nginx
```

📸 **Entregable 2:** Captura del Launch Template mostrando tipo de instancia, SG y metadata:

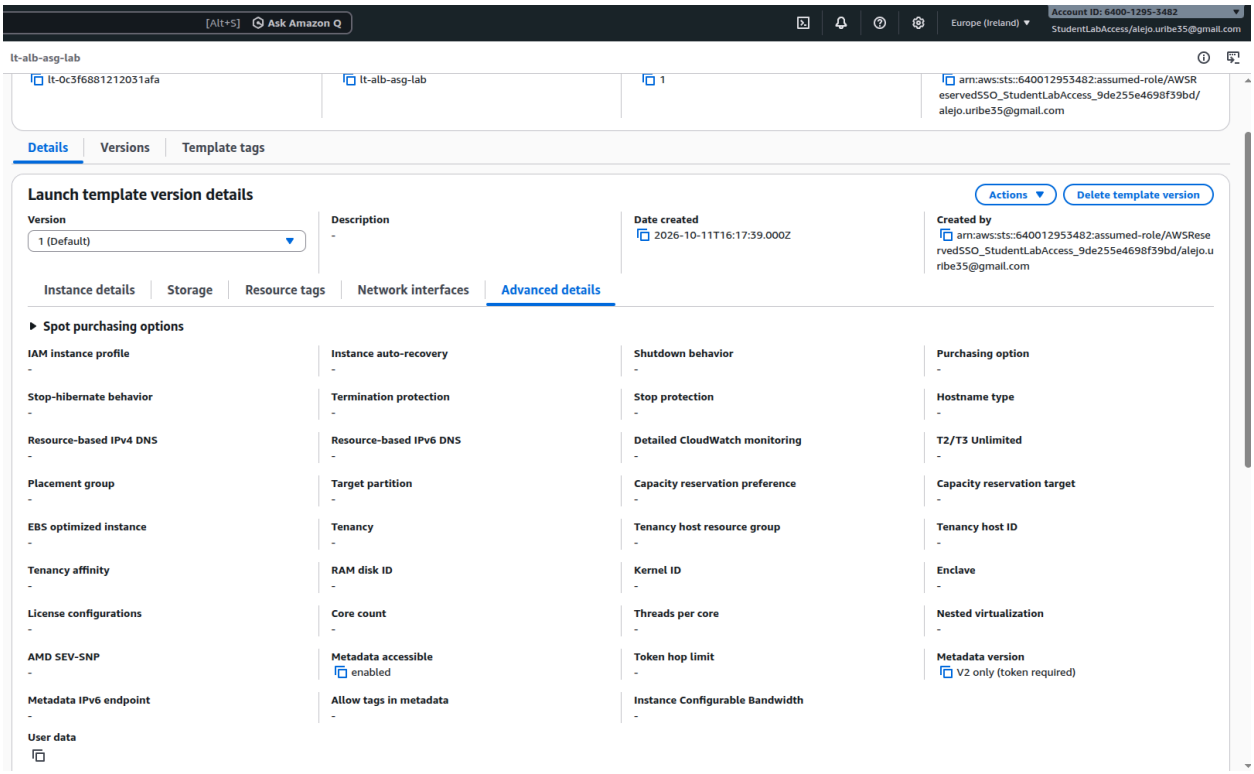
Captura de todo al crearlo



Instance type y Security Group



Recordemos que el SG de security-ec2 el ID era sg-0a6715b45218a4287 que es el mismo que vemos en la captura.

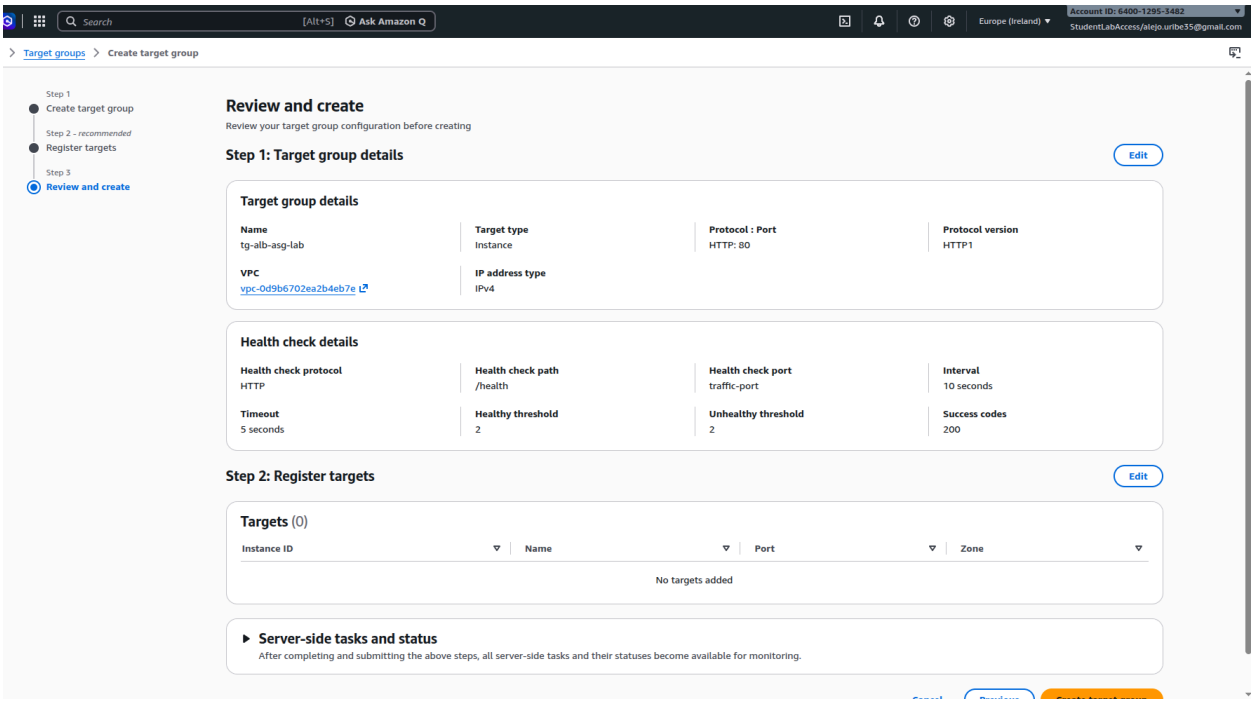


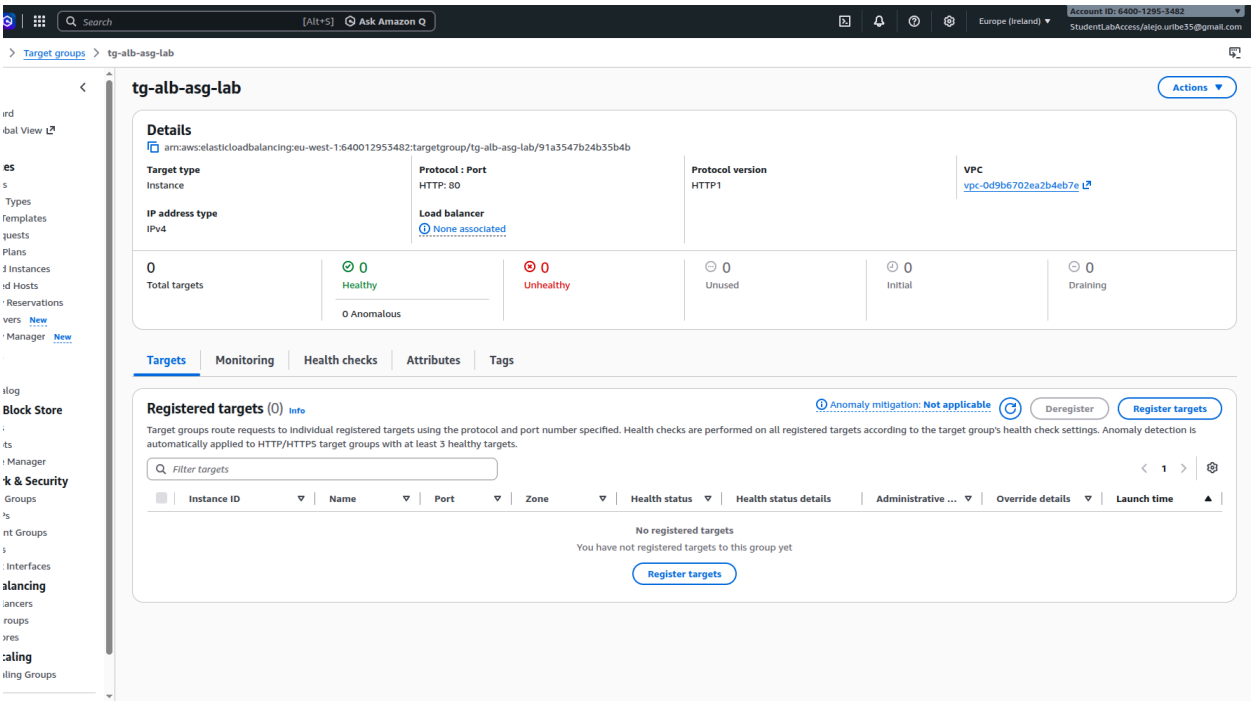
Entregable 3: Target Group

Nombre: `tg-alb-asg-lab`

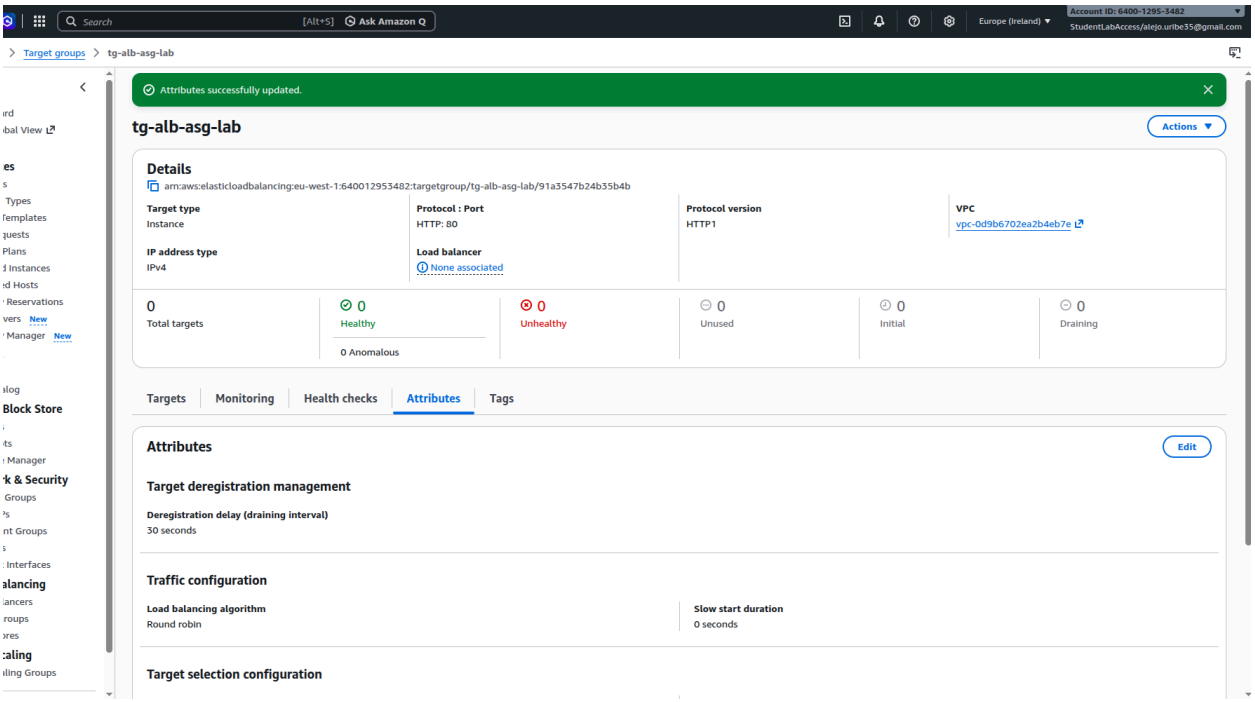
Health checks en /health y con intervalos 10 s, timeout 5 s, healthy threshold 2, unhealthy threshold 2, código de éxito 200, esto lo hacemos para propósitos del laboratorio, en producción el healthy threshold podría ser un poco más alto.

📸 **Entregable 3:** Captura del Target Group creado.

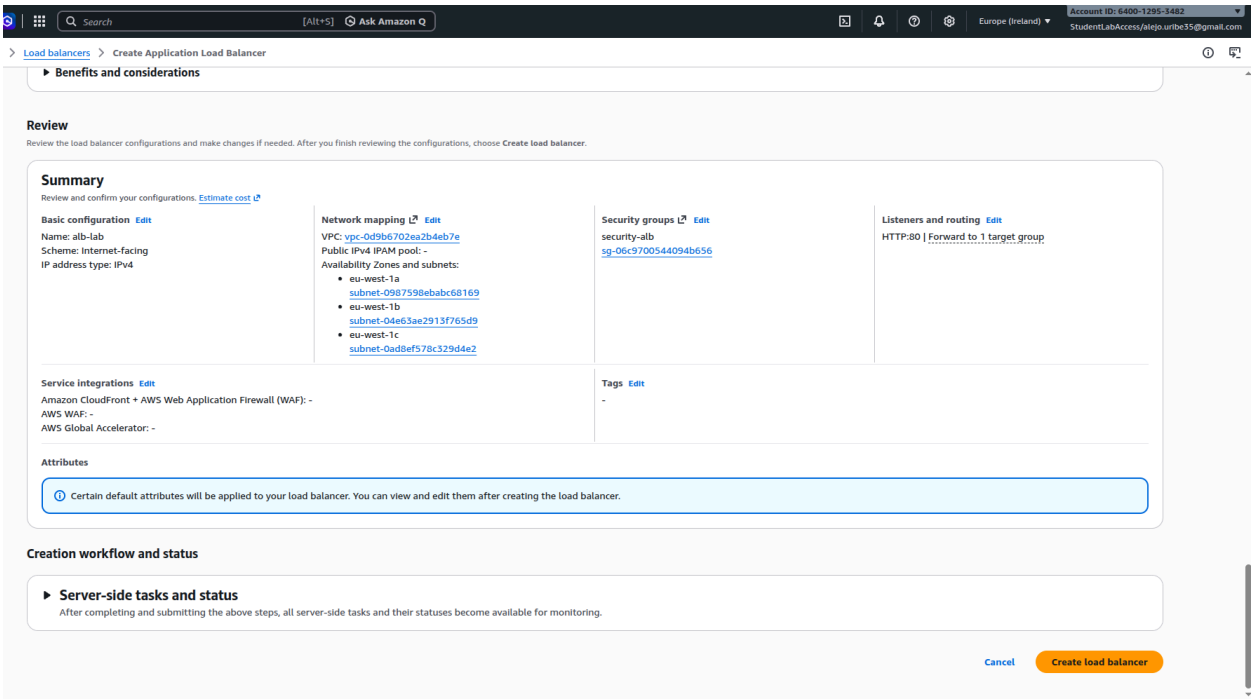




Deregistration delay lo pondré en 120s para que en el entregable 8 sea un poco más rápido, solo por propósitos del laboratorio ya que en producción si se podría dejar con un tiempo mucho más alto.

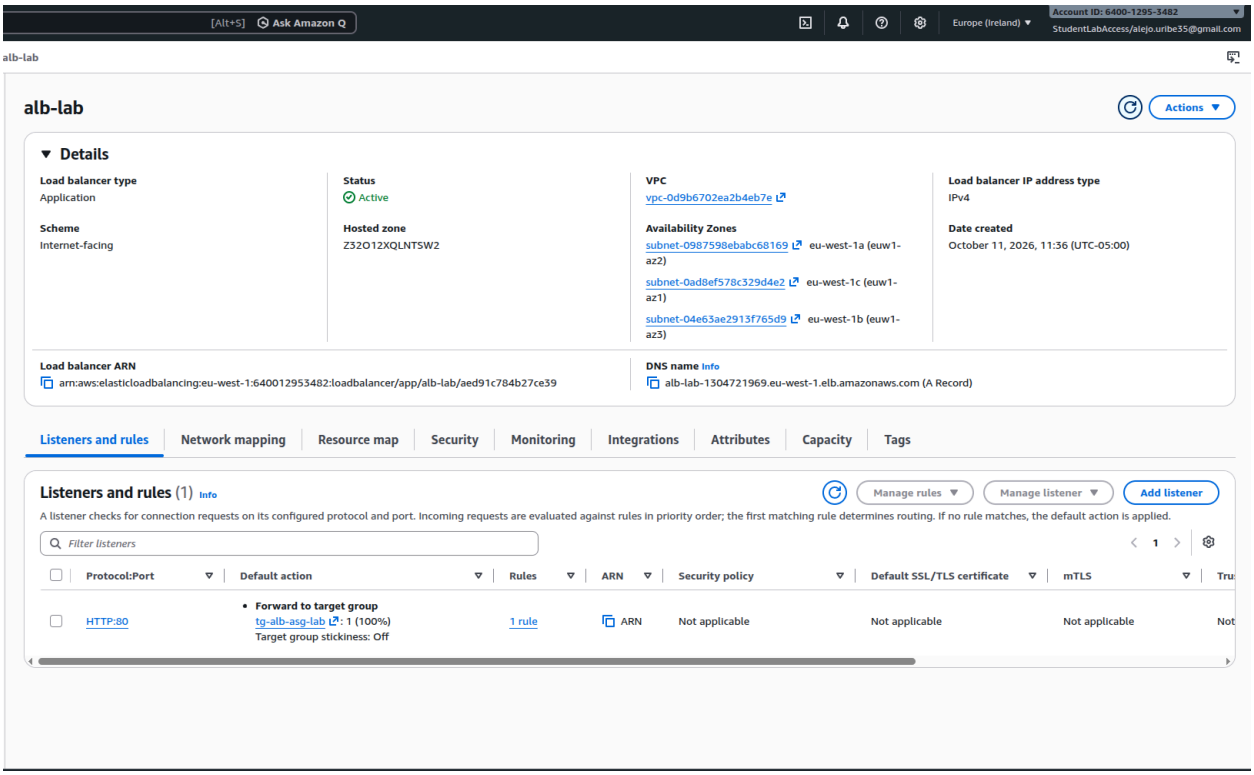


Entregable 4: Crear Application Load Balancer



📸 **Entregable 4:** Captura del ALB mostrando estado activo y DNS Name:

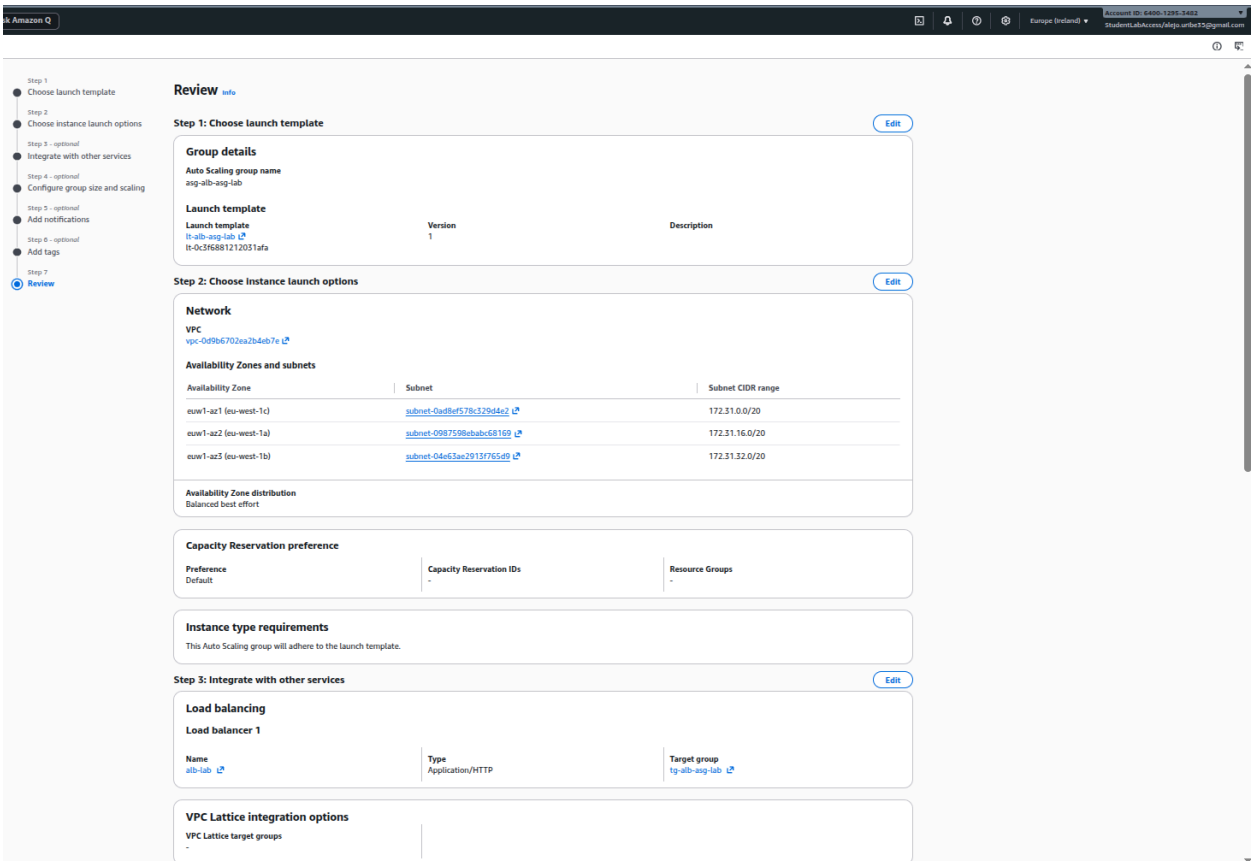
DNS Name: alb-lab-1304721969.eu-west-1.elb.amazonaws.com

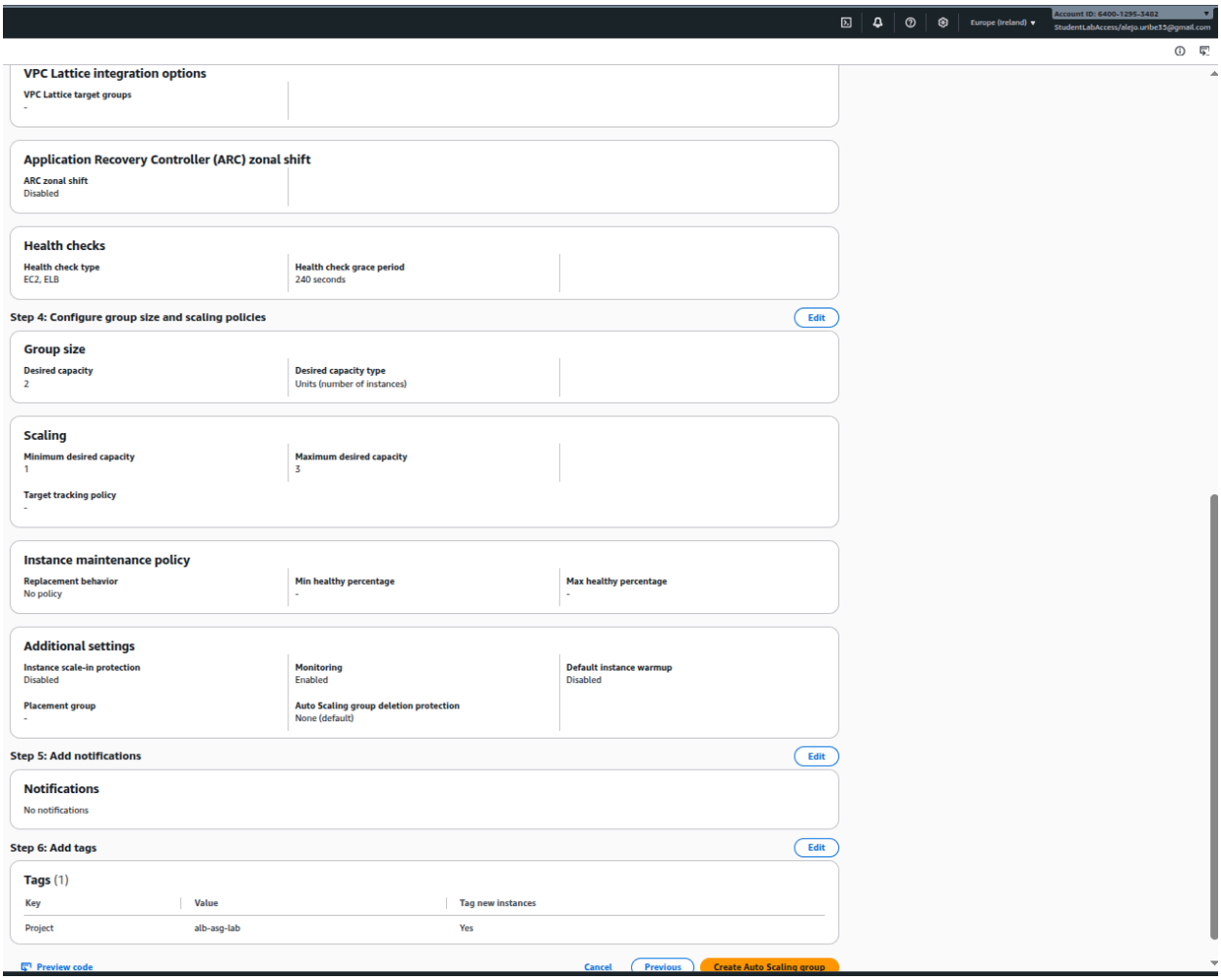


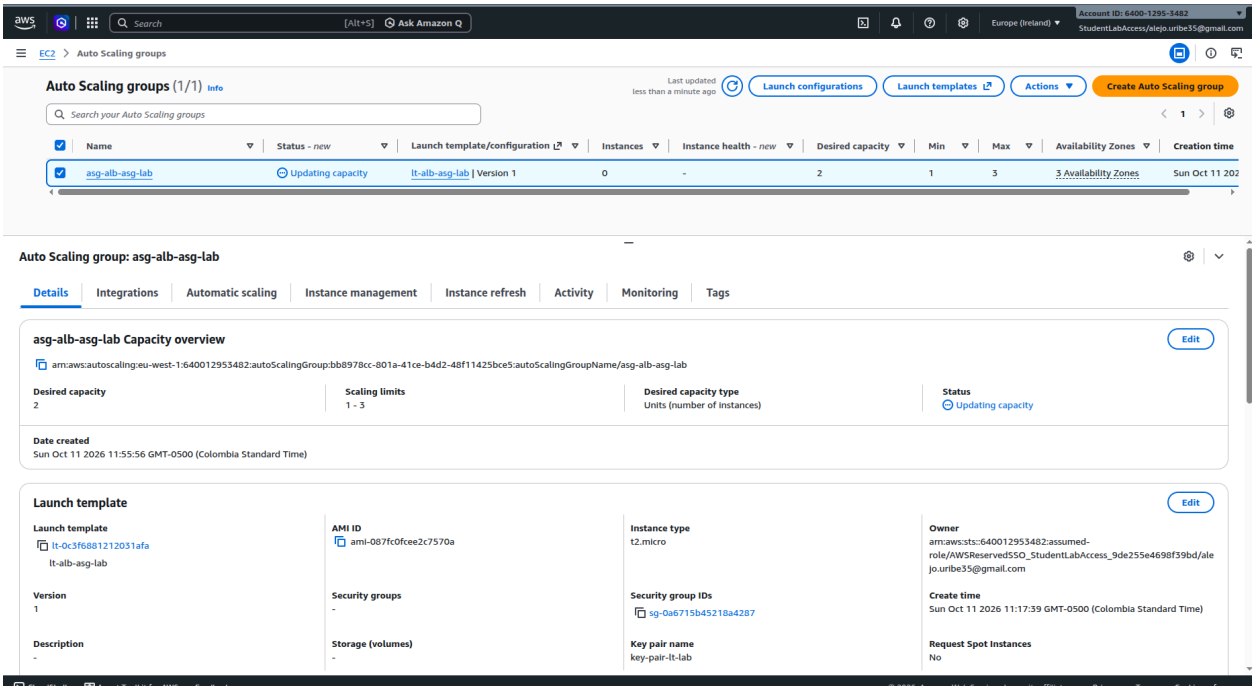
Entregable 5: Crear Auto Scaling Group

Pondremos load balancing health checks, también un periodo de gracia de 240s para estos y las group metrics

📸 **Entregable 5:** Captura mostrando configuración Min / Desired / Max.



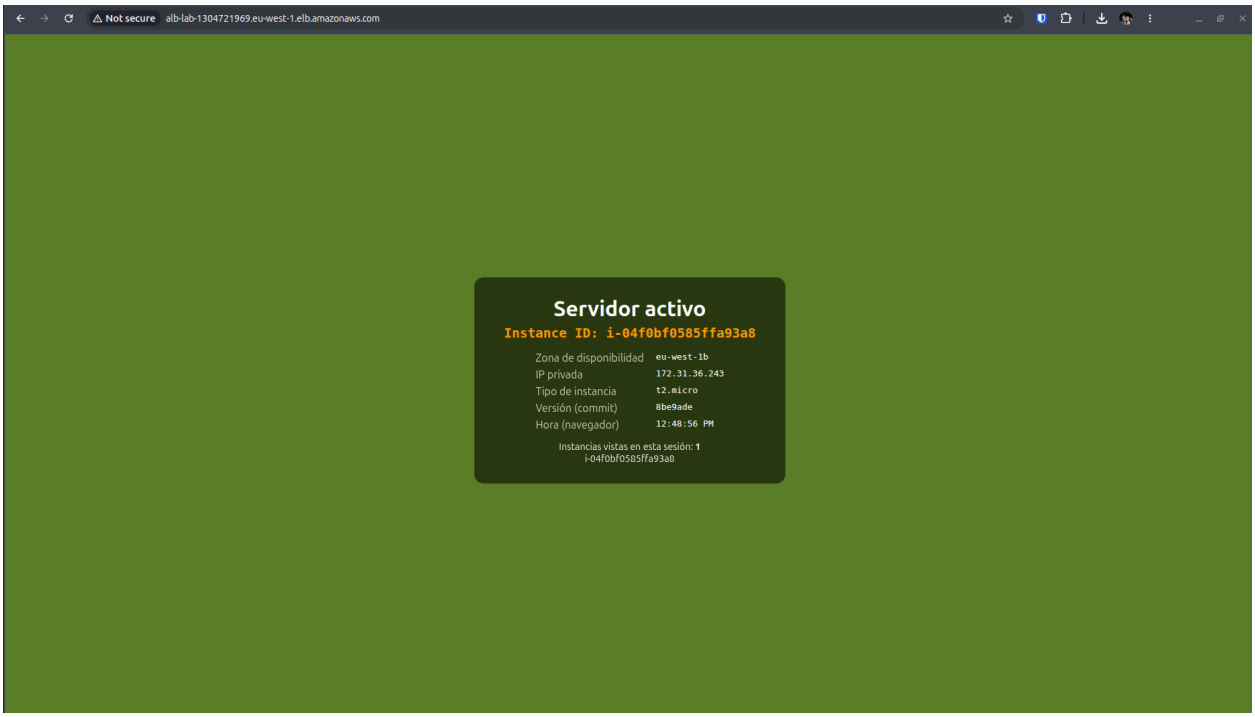

VPC Lattice integration options
VPC Lattice target groups
-
Application Recovery Controller (ARC) zonal shift
ARC zonal shift
Disabled
Health checks
Health check type
EC2, ELB
Health check grace period
240 seconds
Step 4: Configure group size and scaling policies
Edit
Group size
Desired capacity
2
Desired capacity type
Units (number of instances)
Scaling
Minimum desired capacity
1
Maximum desired capacity
3
Target tracking policy
-
Instance maintenance policy
Replacement behavior
No policy
Min healthy percentage
-
Max healthy percentage
-
Additional settings
Instance scale-in protection
Disabled
Monitoring
Enabled
Default instance warmup
Disabled
Placement group
-
Auto Scaling group deletion protection
None (default)
Step 5: Add notifications
Edit
Notifications
No notifications
Step 6: Add tags
Edit
Tags (1)
Key
Value
Tag new instances
Project
alb-asg-lab
Yes
Preview code
Cancel
Previous
Create Auto Scaling group

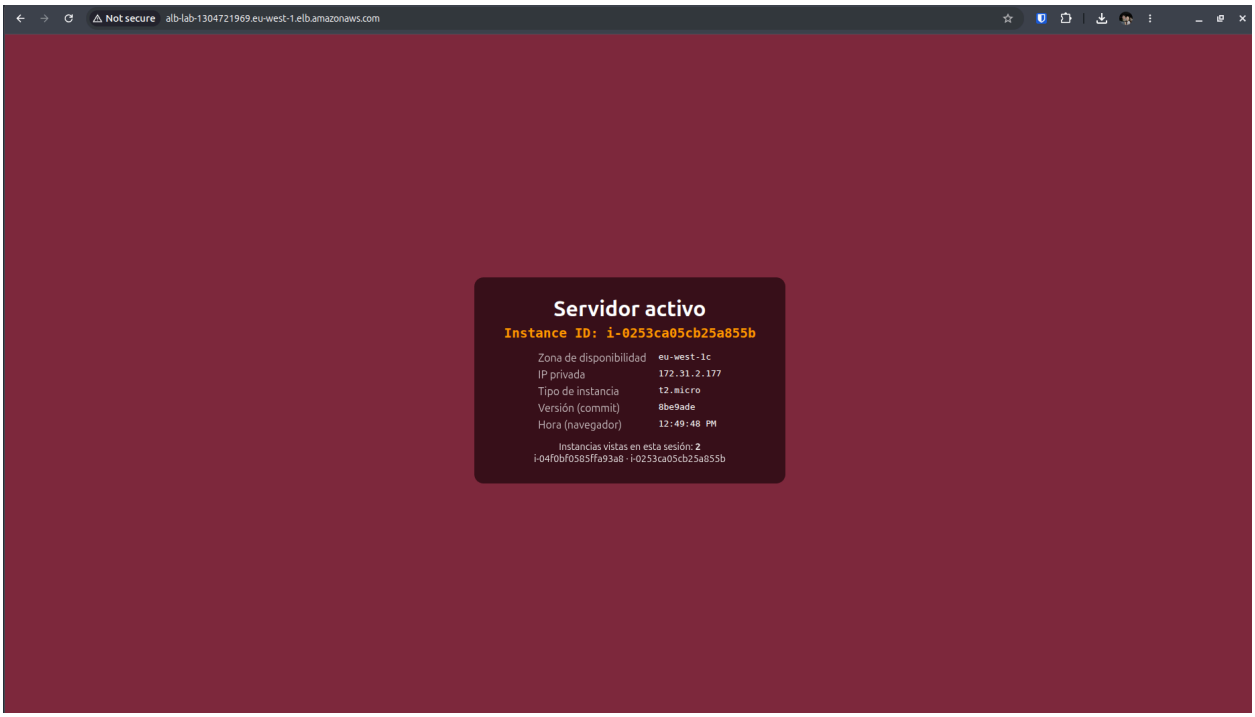

EC2 > Auto Scaling groups
Auto Scaling groups (1/1) Info
Launch configurations
Launch templates
Actions
Create Auto Scaling group
Search your Auto Scaling groups
Name
Status - new
Launch template/configuration
Instances
Instance health - new
Desired capacity
Min
Max
Availability Zones
Creation time
asg-alb-asg-lab
Updating capacity
lt-alb-asg-lab | Version 1
0
2
1
3
3 Availability Zones
Auto Scaling group: asg-alb-asg-lab
Details
Integrations
Automatic scaling
Instance management
Instance refresh
Activity
Monitoring
Tags
asg-alb-asg-lab Capacity overview
Edit
arn:aws:autoscaling:eu-west-1:640012953482:autoScalingGroup:bb8978cc-801a-41ce-b4d2-48f11425bce5:autoScalingGroupName/asg-alb-asg-lab
Desired capacity
2
Scaling limits
1 - 3
Desired capacity type
Units (number of instances)
Status
Updating capacity
Date created
Sun Oct 11 2026 11:55:56 GMT-0500 (Colombia Standard Time)
Launch template
Edit
Launch template
lt-0c3f6881212031afa
lt-alb-asg-lab
AMI ID
ami-087fc0fcee2c7570a
Instance type
t2.micro
Owner
arn:aws:sts::640012953482:assumed-role/AWSReservedSSO_StudentLabAccess_9de255e4698f39bd/alejo.uribe35@gmail.com
Version
1
Security groups
-
Security group IDs
sg-0a6715b45218a4287
Create time
Sun Oct 11 2026 11:17:39 GMT-0500 (Colombia Standard Time)
Description
-
Storage (volumes)
-
Key pair name
key-pair-lt-lab
Request Spot Instances
No

Entregable 6: Probar Balanceo

📸 **Entregable 6:** Dos capturas mostrando IDs diferentes al refrescar.

Como el script cambia entre ellas y hace que refresque la página se puede ver el cambio y los colores ayudan bastante, esta parte queda super clara.





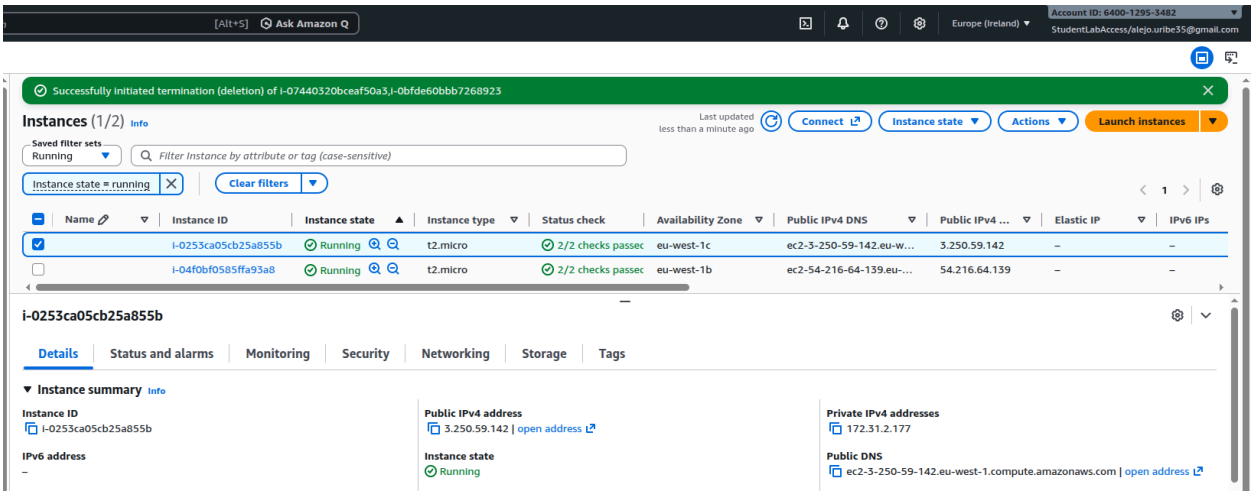
Extra · distribución en números. Desde tu terminal, 20 peticiones y un conteo por instancia:

```
for i in $(seq 1 20); do
  curl -s http://alb-lab-1304721969.eu-west-1.elb.amazonaws.c
om/info.json | grep -o '"instance_id":"[^"]*"'
done | sort | uniq -c
```

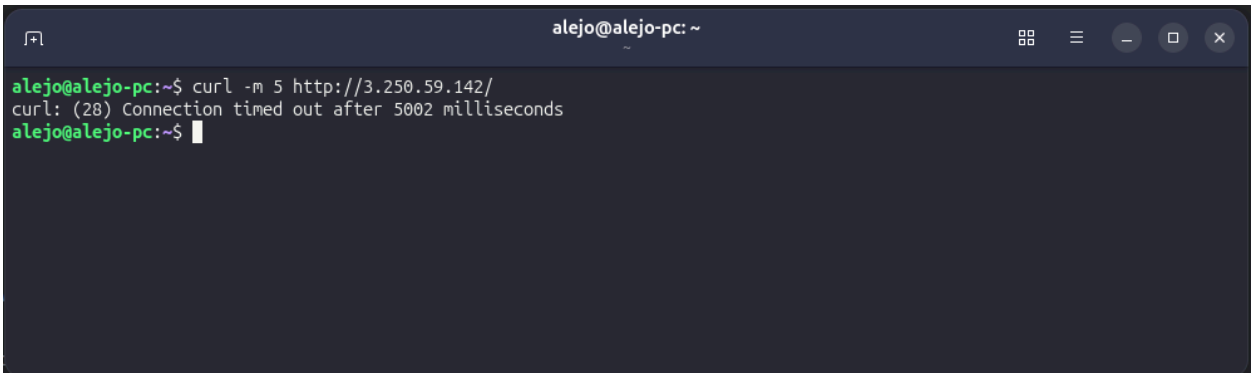


Extra · demostrar el SG chaining. Copia la IP pública de una de las instancias (consola EC2) y prueba el acceso directo: Esto para ver que el SG funciona

correctamente.

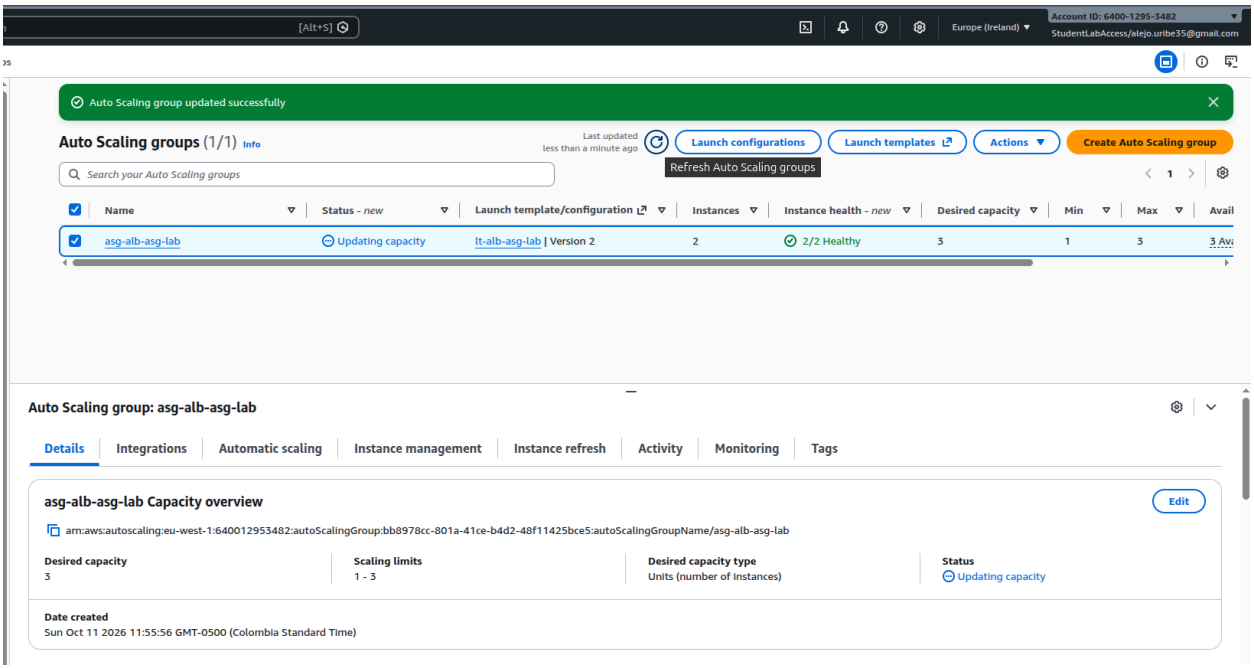


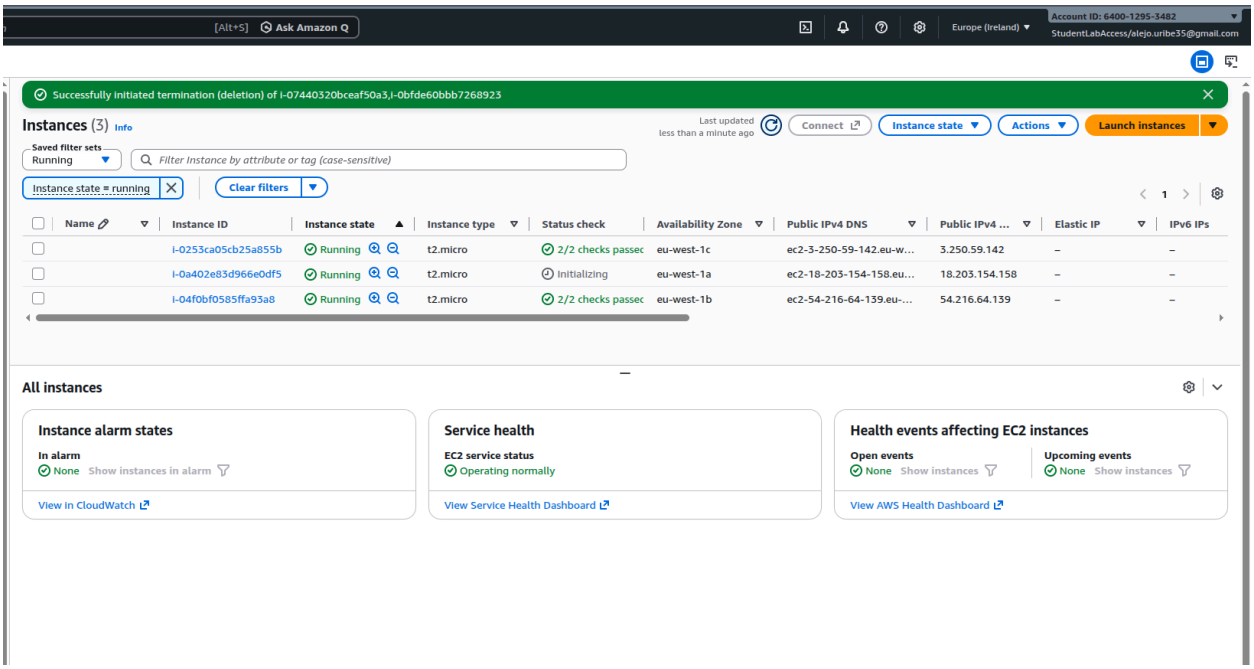

```
curl -m 5 http://3.250.59.142/
```



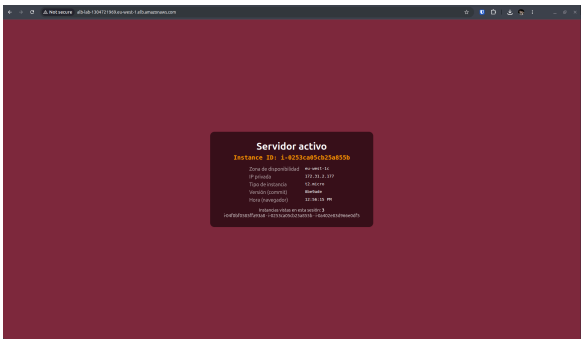
Entregable 7: Probar Escalado

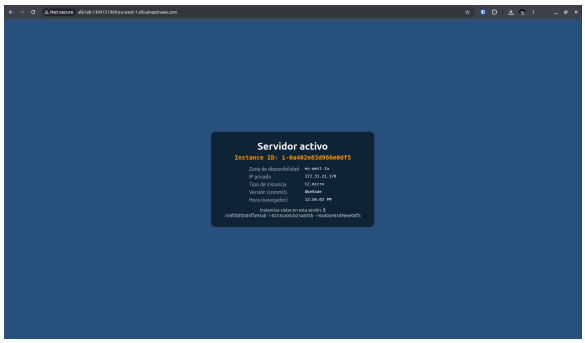
📸 **Entregable 7:** Captura mostrando 3 instancias activas dentro del ASG.

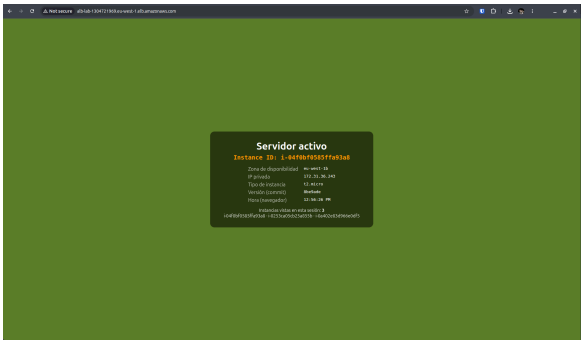


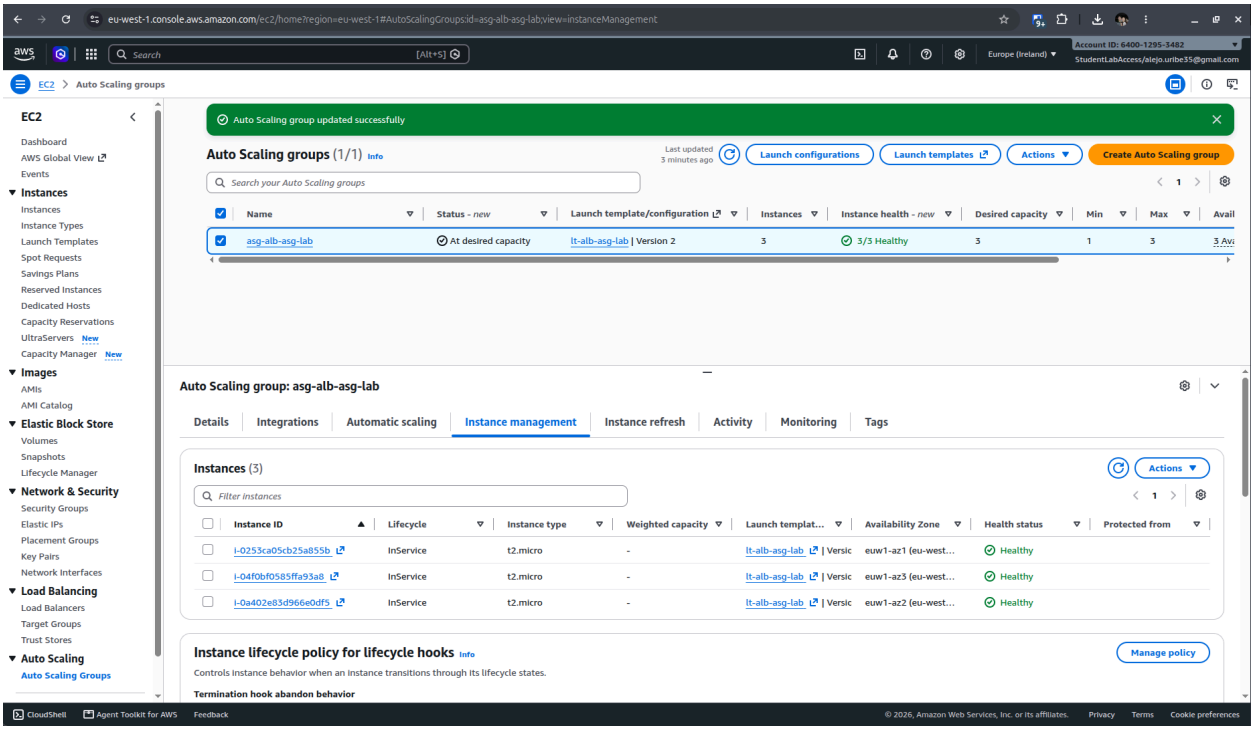


Captura de los 3 servers diferentes con la IP del ALB





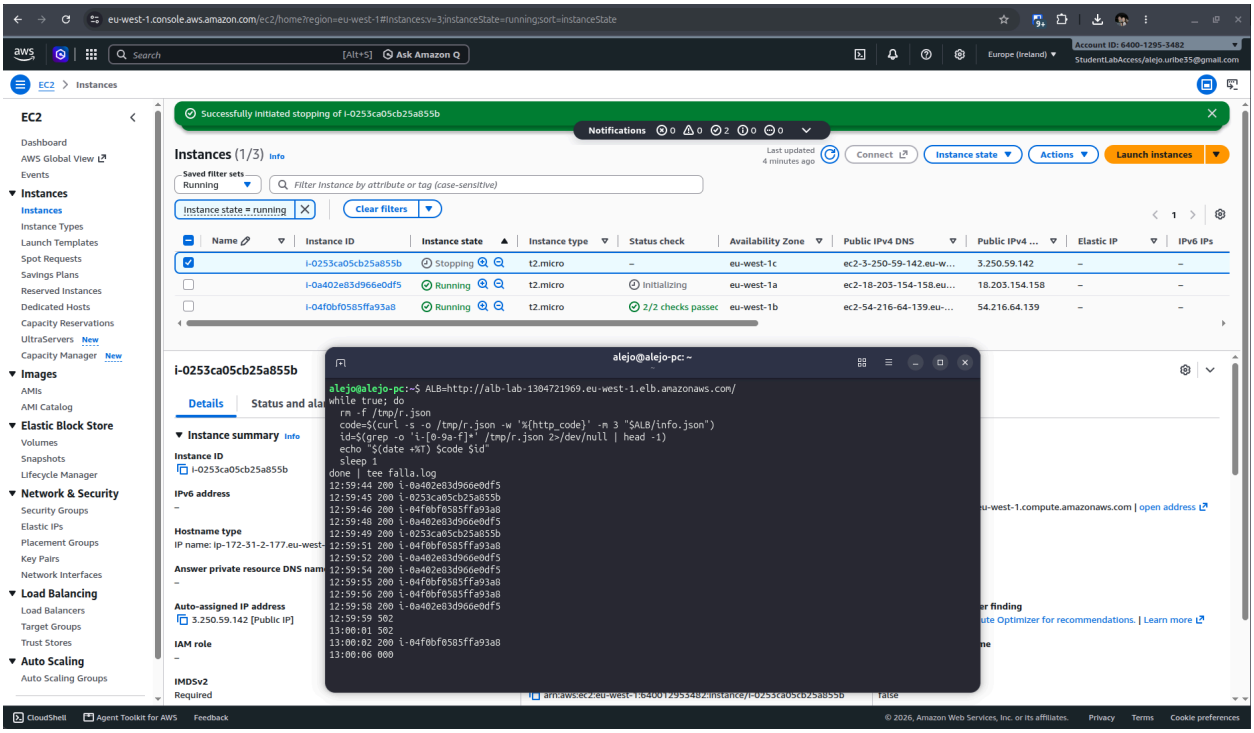


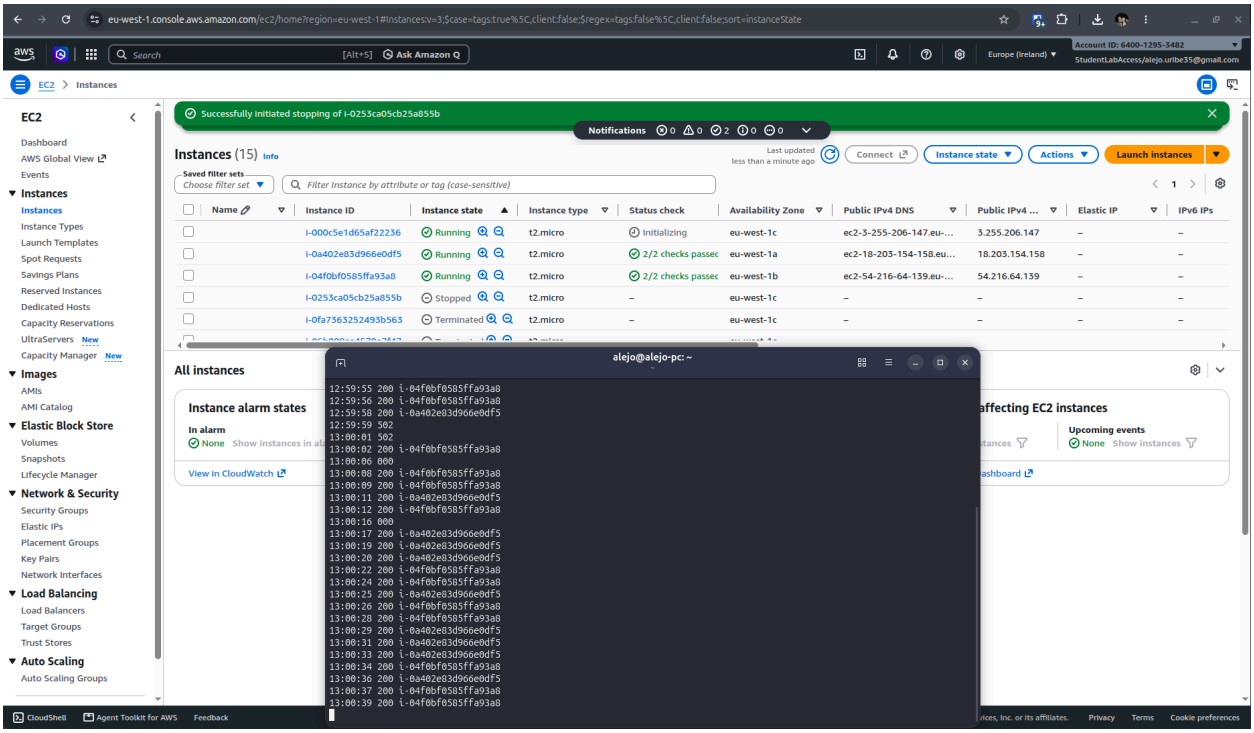


Entregable 8: Probar Alta Disponibilidad

```
ALB=http://alb-lab-1304721969.eu-west-1.elb.amazonaws.com/
while true; do
  rm -f /tmp/r.json
  code=$(curl -s -o /tmp/r.json -w '%{http_code}' -m 3 "$ALB/
info.json")
  id=$(grep -o 'i-[0-9a-f]*' /tmp/r.json 2>/dev/null | head -
1)
  echo "$(date +%T) $code $id"
  sleep 1
done | tee falla.log
```

Mientras se corre el comando detengo una instancia y debería ser capaz de ver por unos momentos errores que son mientras se detecta que la instancia está unhealthy y posteriormente debería volver a funcionar ya que están actuando las otras instancias y se está lanzando una nueva.





falla.log:

```
12:59:44 200 i-0a402e83d966e0df5
12:59:45 200 i-0253ca05cb25a855b
12:59:46 200 i-04f0bf0585ffa93a8
12:59:48 200 i-0a402e83d966e0df5
12:59:49 200 i-0253ca05cb25a855b
12:59:51 200 i-04f0bf0585ffa93a8
12:59:52 200 i-0a402e83d966e0df5
12:59:54 200 i-0a402e83d966e0df5
12:59:55 200 i-04f0bf0585ffa93a8
12:59:56 200 i-04f0bf0585ffa93a8
12:59:58 200 i-0a402e83d966e0df5
12:59:59 502
13:00:01 502
13:00:02 200 i-04f0bf0585ffa93a8
13:00:06 000
13:00:08 200 i-04f0bf0585ffa93a8
13:00:09 200 i-04f0bf0585ffa93a8
```

```
13:00:11 200 i-0a402e83d966e0df5
13:00:12 200 i-04f0bf0585ffa93a8
13:00:16 000
13:00:17 200 i-0a402e83d966e0df5
13:00:19 200 i-0a402e83d966e0df5
13:00:20 200 i-0a402e83d966e0df5
13:00:22 200 i-04f0bf0585ffa93a8
13:00:24 200 i-04f0bf0585ffa93a8
13:00:25 200 i-0a402e83d966e0df5
13:00:26 200 i-04f0bf0585ffa93a8
13:00:28 200 i-04f0bf0585ffa93a8
13:00:29 200 i-0a402e83d966e0df5
13:00:31 200 i-0a402e83d966e0df5
```

Fase	Dónde la ves	Hora	Segundos desde la detención
Detienes la instancia	tu reloj	12:59:55	0s
Primer error en el monitor	`falla.log`	12:59:59	4s
El target pasa a *unhealthy* o *draining*	Target Group → Targets	13:00:05	10s
El ASG lanza el reemplazo	ASG → Activity	13:00:18	25s
El reemplazo pasa a *healthy*	Target Group → Targets	13:00	30s ? No me da segundos
Últimos errores en el monitor	`falla.log`	13:00:16	23s

Preguntas finales:

- Breve explicación (10–12 líneas) respondiendo:
    1. ¿Qué función cumple el Application Load Balancer?
        a. Sería el punto de entrada a mis servidores, ya que todo el tráfico web pasa por el ALB antes de entrar a mis servidores, de hecho el ALB es el que entra a los servidores, este repoarte las peticiones entre las

instancias que están en el Target Group, este también se encarga de hacer los health checks para revisar que cada instancia funcione correctamente y si alguna no se encuentra disponible no envía tráfico a ella hasta que todo se encuentre bien.

2. ¿Qué hace el Auto Scaling Group?
    a. Básicamente escala los servidores, tanto para arriba como para abajo, es decir, crea o elimina instancias a partir de un Launch Template dependiendo de las reglas definidas, puede ser por consumo de CPU o diferentes medidas, se le define una capacidad mínima, máxima y deseada para que este pueda crear o disminuir las instancias dependiendo de lo que se necesite en cada momento.
3. ¿Qué diferencia hay entre IMDSv1 e IMDSv2?
    a. v1 no necesita autenticación, mientras que la v2 necesita de un token que se pide con un método PUT y luego con ese token se pueden hacer las peticiones necesarias para obtener los metadatos.
4. ¿Qué ocurre cuando detienes una instancia?
    a. Está se vuelve unhealthy en el Target Group y en la actividad del ASG se puede ver que también se da cuenta de que esa instancia ya no está en servicio por los mismos health checks entonces como se definió una capacidad deseada, en este caso, de 3 instancias entonces lo que hace al detener una es que se crea una nueva hasta alcanzar ese número deseado de intancias, el servicio sigue funcionando con las otras instancias pero puede haber una ventana corta de errores como podemos ver en la anterior tabla.

Al final tuve algunos errores, el script falló al inicio el de github de mi repositorio, me tocó cambiarlo, también cambiar el user data, me tocó crear una nueva versión de launch template creado, saqué la versión 2 pero como no había cambiado la versión en el ASG entonces me seguía sacando instancias con la versión 1 y el error persistía pero aprendí varias cositas entre conexiones de estos servicios.